Problem 1 (3 pts)

Consider the following matrix:

$$C = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 5 & 7 & 9 \\ 3 & 7 & 11 & 14 \\ 4 & 9 & 14 & 19 \end{bmatrix}.$$

Write down the Cholesky factorization of the above matrix.

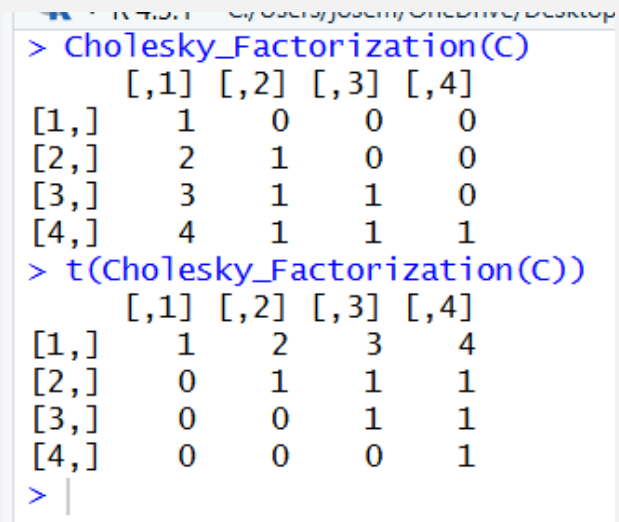

Figure 1: Cholesky Factorization Algorithm Output

While a code was not required I have provided one as my work. See (code appendix). Since the matric C is both symmetric and positive definite the Cholesky Factorization exists, and is given below.

$$\begin{aligned} C &= LL^{\mathsf{T}} \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 1 & 0 & 0 \\ 3 & 1 & 1 & 0 \\ 4 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \end{aligned}$$

Problem 2 (3 pts)

Computational problem. Consider the following matrix:

$$A = \begin{bmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

Use the Power method to find the leading eigenvalue and the corresponding eigenvectors, starting with

$$x_0 = (1, -1, 2)^\mathsf{T}.$$

```
> PM(A,x0)
$vec
          [,1]
[1,] 0.5773503
[2,] 0.5773502
[3,] 0.5773503

$val
[1] 4
```

Figure 2: Power Method Algorithm Output

Above we see the results of my power method algorithm implementation. In particular we see that the leading eigenvalue is $\lambda = 4$ and has a corresponding eigenvector $\frac{1}{\sqrt{3}}(1, 1, 1)$. For code details see (code appendix).

Problem 3 (4 pts)

For a given vector $v \in \mathbb{R}^n$, $||v|| = 1$. Consider the associated Householder reflection matrix

$$P = I - 2vv^\mathsf{T}.$$

Show that $P^\mathsf{T} = P$ and $P^2 = I$.

(i) First we show that $P^\mathsf{T} = P$. By definition we get,

$$
\begin{aligned}
P^\mathsf{T} &= (I - 2vv^\mathsf{T})^\mathsf{T} \\
&= I^\mathsf{T} - 2(vv^\mathsf{T})^\mathsf{T} \quad \text{(transpose is a linear transformation)} \\
&= I - 2(v^\mathsf{T})^\mathsf{T} v^\mathsf{T} \quad (AB^\mathsf{T} = B^\mathsf{T} A^\mathsf{T}) \\
&= I - 2vv^\mathsf{T} = P \\
\therefore P^\mathsf{T} &= P.
\end{aligned}
$$

(ii) Now we show that P is involutory.

$$
\begin{aligned}
P^2 &= (I - 2vv^\mathsf{T})(I - 2vv^\mathsf{T}) \\
&= I^2 - 4vv^\mathsf{T} + 4(vv^\mathsf{T})^2 \\
&= I - 4vv^\mathsf{T} + 4(vv^\mathsf{T})(vv^\mathsf{T}) \\
&= I - 4vv^\mathsf{T} + v(v^\mathsf{T} v)v^\mathsf{T} \\
&= I - 4vv^\mathsf{T} + v(||v||^2)v^\mathsf{T} \\
&= I - 4vv^\mathsf{T} + vv^\mathsf{T} \quad (\text{using } ||v|| = 1) \\
\therefore P^2 &= I.
\end{aligned}
$$

Problem 4 (6 pts)

Computational problem. Consider a matrix

$$
A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 9 & 7 \\ 3 & 9 & 1 & 1 \\ 4 & 7 & 1 & 10 \end{bmatrix}.
$$

Notice that $A = A^\mathsf{T}$. Write a code of the QR iteration, following the 2-step algorithm. That is,

Step 1: use the Householder reflections to transform the original matrix A into an upper Hessenberg matrix H.
Step 2: Perform the QR iterations on H, with the QR factorization can be obtained through rotations.
Apply your QR iteration code to the given matrix A. Let your code print out the following matrices:

(a) The initial upper Hessenberg matrix H, let it be $H^{(0)}$.

(b) The next $H^{(1)}$.

(c) Finally, the matrices $\{H^{(k)}\}$ converges to some diagonal matrix *. Print H^*.

(d) All eigenvalues of H^* are also the eigenvalues of the original matrix A.

```
> # part a : H^{(0)}
> H <- Hessenberg(test_matrix)
> H
          [,1]      [,2]       [,3]       [,4]
[1,]  1.000000 -5.385165  0.0000000  0.0000000
[2,] -5.385165 14.793103 -7.1841747  0.0000000
[3,]  0.000000 -7.184175 -5.1437923 -0.9619999
[4,]  0.000000  0.000000 -0.9619999  5.3506888
> # part b : H^{(1)}
> H1 <- round(QR_step(H), 10)
> H1
          [,1]      [,2]      [,3]      [,4]
[1,] 16.266667 -7.509697  0.000000  0.000000
[2,] -7.509697 -5.443119  1.025482  0.000000
[3,]  0.000000  1.025482  3.855825 -2.476109
[4,]  0.000000  0.000000 -2.476109  1.320627
> # part c : H^{*}
> round(QR_algorithm(test_matrix, 2000),10)
         [,1]      [,2]     [,3]       [,4]
[1,] 18.61768  0.000000 0.000000  0.0000000
[2,]  0.00000 -7.874269 0.000000  0.0000000
[3,]  0.00000  0.000000 5.417752  0.0000000
[4,]  0.00000  0.000000 0.000000 -0.1611592
>
> # part d : Verification of eigenvalues
> eigen(test_matrix)$val
[1] 18.6176762  5.4177524 -0.1611592 -7.8742694
>
```

Figure 3: QR Iteration Results

Using my QR algorithm see (fig. 3 and code appendix), I got the following answers.

(a)

$$H^{(0)} = \begin{bmatrix} 1 & -5.385 & 0 & 0 \\ -5.385 & 14.793 & -7.184 & 0 \\ 0 & -7.184 & -5.143 & -0.962 \\ 0 & 0 & -0.962 & 5.351 \end{bmatrix}$$

(b)

$$H^{(1)} = \begin{bmatrix} 16.267 & -7.510 & 0 & 0 \\ -7.510 & -5.443 & 1.025 & 0 \\ 0 & 1.025 & 3.856 & -2.476 \\ 0 & 0 & -2.476 & 1.321 \end{bmatrix}$$

(c)

$$H^{(k)} \to H^* = \begin{bmatrix} 18.618 & 0 & 0 & 0 \\ 0 & -7.874 & 0 & 0 \\ 0 & 0 & 5.418 & 0 \\ 0 & 0 & 0 & -0.161 \end{bmatrix}$$

(d) Using R's built in eigenvalue function we see that the eigenvalues match up with the diagonal entries of H^*, see fig. 3.

Problem 5 (4 pts)

Theorem. Consider $A = P - R$ where P is invertible, and write $B := P^{-1}R$. Consider the iteration

$$x_{n+1} = Bx_n + P^{-1}b.$$

Suppose $||B|| < 1$ for some operator/induced norm $|| \cdot ||$ over $(\mathbb{R}^n, || \cdot ||)$, i.e.

$$||B|| = \sup_{x \in V} \frac{||Bx||}{||x||}.$$

Then there exists a unique x^* such that for all $x_0 \in V$, x_n converges to x^*. In particular x^* satisfies

$$Ax^* = b.$$

Moreover,

$$||x^* - x_n|| \leq \frac{||B||^n}{1 - ||B||} ||x_1 - x_0||.$$

First we will prove uniqueness and then show the error bound holds.

(i) Suppose that $||B|| < 1$. Then there exists an x^* such that under the iteration,

$$x_{n+1} = Bx_n + P^{-1}b \tag{*}$$

$x_n \to x^*$ and $Ax^* = b$.

To show this first define the iteration mapping $T(x) : \mathbb{R}^n \to \mathbb{R}^n$ with rule

$$T(x) = Bx + P^{-1}b.$$

Then for $x, y \in \mathbb{R}^n$ consider

$$||T(x) - T(y)|| = ||B(x - y)|| \leq ||B|| ||x - y||.$$

Since $||B|| < 1$ we know that T is a contraction mapping and there is a unique fixed point, we will call it x^*. Then we know that $x^* = T(x^*)$. We get

$$\begin{aligned} x^* &= Bx^* + P^{-1}b \\ \implies Px^* &= P(P^{-1}R)x^* + b \\ \implies (P-R)x^* &= b \\ \implies Ax^* &= b. \end{aligned}$$

Such an x^* must be unique by the Banach-Fixed Point theorem.

(ii) Consider the following norm,

$$\begin{aligned} ||x_{n+1} - x_n|| &= ||T(x_n) - T(x_{n-1})|| \\ &= ||B(x_n - x_{n-1})|| \\ &= ||B|| \cdot ||x_n - x_{n-1}|| \\ &= ||B||^n ||x_1 - x_0|| \quad \text{(by induction)} \\ \implies ||x_{n+1} - x_n|| &= q^n ||x_1 - x_0|| \qquad (*) \end{aligned}$$

where $q := ||B|| < 1$. Now consider the error bound we are after,

$$\begin{aligned} x^* - x_n &= \sum_{k=n}^{\infty} (x_{k+1} - x_k) \quad \text{(telescoping upwards)} \\ \implies ||x^* - x_n|| &= \left|\left| \sum_{k=n}^{\infty} (x_{k+1} - x_k) \right|\right| \\ &\leq \sum_{k=n}^{\infty} ||x_{k+1} - x_k|| \quad \text{(triangle inequality)} \\ &= \sum_{k=n}^{\infty} q^k \, ||x_1 - x_0|| \quad \text{(using } (*)\text{)} \\ &= \frac{q^n}{1-q} \, ||x_1 - x_0|| \quad \text{(geometric series)} \end{aligned}$$

Substituting back in for q we get,

$$\boxed{||x^* - x_n|| \leq \frac{||B||^n}{1 - ||B||} \, ||x_1 - x_0||}$$

Code Appendix

Algorithms are written in R.

Cholesky Factorization Algorithm

```
install.packages("matlib")
library(matlib)

## swap row i with row j
elem1 <- function(A, i, j){
  e <- diag(nrow(A))
  c <- rowswap(e, from = i, to = j)
  return(c)
}

## scale row i with scalar s
elem2 <- function(A, i, s){
  e <- diag(nrow(A))
  e[i,] <- s*e[i,]
  return(e)
}

## to row i add s*row j
elem3 <- function(A, i, s, j){
  e <- diag(nrow(A))
  e[i,] <- e[i,] + s * e[j,]
  return(e)
}

## REF algorithm
ref <- function(A){
  e_list <- list()
  inv_list <- list()
  N <- nrow(A)
  M <- ncol(A)
  B <- A

  for (j in 1:min(N,M)) {

    ## find pivot in column j
    idx <- NA
    for (i in j:N) {
      if (B[i, j] != 0) {
        idx <- i
        break
      }
    }

    ## if entire column below is zero, move on
    if (is.na(idx)) next

    ## swap pivot row into position j
    E1 <- elem1(B, j, idx)
    B <- E1 %*% B
```

```r
    e_list[[length(e_list) + 1]] <- E1
    inv_list[[length(inv_list) + 1]] <- E1

    ## eliminate entries below pivot
    if (j < N) {
      for (i in (j + 1):N) {

        if (B[i, j] == 0) next

        s <- - B[i, j] / B[j, j]
        E3 <- elem3(B, i, s, j)
        inv_E3 <- elem3(B, i, -s, j)

        B <- E3 %*% B
        e_list[[length(e_list) + 1]] <- E3
        inv_list[[length(inv_list) + 1]] <- inv_E3
      }
    }
  }

  L <- Reduce(`%*%`, inv_list)

  return(list(
    b = B,
    actions = e_list,
    inv_actions = inv_list,
    l = L
  ))
}


LU_Decomp <- function(A){
  L <- ref(A)$l
  U <- ref(A)$b
  return(
    list(
      l = L, u = U
    )
  )
}

LDU_Decomp <- function(A){
  L <- ref(A)$l
  U_til <- ref(A)$b

  d <- c()
  d_inv <- c()
  for(i in 1:ncol(U_til)){
    d <- c(d, U_til[i,i])
    d_inv <- c(d_inv, 1/(U_til[i,i]))
  }
  D <- diag(d)
  D_inv <- diag(d_inv)
  U <- D_inv %*% U_til

  return(list(
    l = L, d = D, u = U, a = L %*% D %*% U
```

```
  ))
}

Cholesky_Factorization <- function(A){
  if (identical (A, t(A))){
    if(det(A) != 0){
      L <- LDU_Decomp(A)$l
      D <- LDU_Decomp(A)$d
      U <- LDU_Decomp(A)$u

      D_bar <- sqrt(D)
      Chol <- L %*% D_bar ## cholesky factorization
      return(Chol)
    }
    else("Matrix input is singular")
  }
  else(print("Matrix input is not symmetric"))
}

C = matrix(c(1,2,3,4,2,5,7,9,3,7,11,14,4,9,14,19), ncol = 4,
           byrow = TRUE)

Cholesky_Factorization(C)
t(Cholesky_Factorization(C))
```

Power Method Algorithm

```
## Power Method Algorithm ##

PM <- function(A, x_init, tol = 1e-6, max_steps = 500){
  x_old <- x_init

  i <- 1
  while(i <= max_steps){
    x_new <- (A %*% x_old) / norm(A %*% x_old, type="2")
    i <- i + 1
    if(norm(x_new - x_old, type="2") < tol){
      break
    }
    x_old <- x_new
  }
  return(list(
    vec = x_new, val = (A%*%x_new)[1]/(x_new[1])
  ))
}

## Given SPD Matrix
A <- matrix(c(2,1,1,1,2,1,1,1,2), byrow = TRUE, ncol = 3)
## Given initial vector
x0 <- matrix(c(1,-1,2), nrow = 3)
## Calling Power Method Algorithm
PM(A,x0)
```

QR Iteration Algorithm

```r
## QR Iteration Algorithm

## Turns a square matrix into an upper Hessenberg matrix
Hessenberg <- function(A){
  ## check for square matrix
  if(nrow(A) == ncol(A)){
    N <- nrow(A)
    H <- A
    for(i in 1:(N-2)){
      x <- as.matrix(H[(i+1):N, i])
      e1 <- c(1, rep(0, length(x)-1))

      q <- x + sign(x[1])*norm(x,"2") * e1
      q <- q / (norm(q, type="2"))
      Psmall <- diag(length(q)) - 2*q %*% t(q)

      P <- diag(N)
      P[(i+1):N, (i+1):N] <- Psmall


      H <- P %*% H %*% P
    }
    rtol <- 1e-12
    H[abs(H) < rtol] <- 0
    return(H)
  }

  else{
    print("Matrix input is not square")
  }
}

## Runs one QR step on an arbitrary matrix
QR_step <- function(H){
  N <- ncol(H)

  for (i in 1:(N-1)){

    v <- H[c(i,i+1),i]
    r <- norm(v, "2")
    if (r == 0) next

    c <- v[1] / r
    s <- v[2] / r

    BR <- diag(N)
    BR[c(i, i+1), c(i, i+1)] <- matrix(c(c, s, -s, c), nrow = 2)

    H <- t(BR) %*% H %*% BR
  }
  return(H)
}

## Takes arb. matrix -> Hessenberg -> Runs many QR steps
```

```r
QR_algorithm <- function(A, maxiter = 500, rtol = 1e-12){
  H <- Hessenberg(A)
  for (k in 1:maxiter){
    H <- QR_step(H)
  }
  tol <- rtol
  H[abs(H) < tol] <- 0
  return(H)
}

## test matrix from HW
test_matrix <- matrix(c(1,2,3,4,2,4,9,7,3,9,1,1,4,7,1,10), ncol = 4)

# part a : H^{(0)}
H <- Hessenberg(test_matrix)
H
# part b : H^{(1)}
H1 <- round(QR_step(H), 10)
H1
# part c : H^{*}
round(QR_algorithm(test_matrix, 2000),10)

# part d : Verification of eigenvalues
eigen(test_matrix)$val
```